

Università degli Studi dell'Insubria
Dipartimento di Scienze Teoriche e Applicate
Laboratorio Interdisciplinare A — a.a. 2025/2026

Cine-Max

Sistema di Gestione Cinema

MANUALE TECNICO

Nome	Matricola	Contributo principale
Serrao Isabella	766930 VA	TUI, Service layer, GitHub, Documentazione
Wilson Bernal Gaia	766733 VA	Persistence CSV (ProiezioneRepository, UtenteRepository, FileManager)
Rahmouni Malek	766890 VA	Persistence CSV (PrenotazioneRepository, FileManager)
Macri Pornima	765952 VA	Model layer (Film, Proiezione, Prenotazione, Utente e sottoclassi, Ruolo)

INDICE

Indice.....	Errore. Il segnalibro non è definito.
1. Installazione	4
1.1 Requisiti di sistema	4
1.2 Setup ambiente Eclipse.....	4
1.3 Compilazione da riga di comando.....	4
1.4 Creazione del JAR eseguibile.....	5
1.5 Esecuzione.....	5
2. Architettura dell'applicazione	6
2.1 Scelta architetturale: architettura a livelli	6
2.2 Struttura del package cinemax	6
3. Strutture dati.....	7
3.1 Classi del Model layer	7
Gerarchia Utente	7
Film.....	7
Proiezione	7
Prenotazione	8
3.2 Strutture dati in memoria nel GestoreApp	8
4. Scelte algoritmiche.....	9
4.1 Cifratura delle password (SHA-256).....	9
4.2 Controllo sovrapposizione proiezioni	9
4.3 Ricerca con filtri combinabili.....	9
4.4 Codici univoci con UUID	10
5. Formato dei file CSV e gestione della persistenza	11
5.1 Formato dei file.....	11
utenti.csv.....	11
proiezioni.csv	11
prenotazioni.csv	11
5.2 Gestione dei percorsi	12
5.3 Classe FileManager	12
5.4 Strategia di persistenza nel GestoreApp.....	12
6. Pattern e scelte implementative.....	13
6.1 Pattern utilizzati	13
Singleton implicito — GestoreApp.....	13
Factory implicita — costruttori specializzati.....	13
Classi utility non istanziabili	13
Template Method implicito — gerarchia Utente.....	13

6.2 Scelte implementative rilevanti	13
Enum Ruolo	13
Uso di Java Streams	13
Encoding UTF-8 esplicito	13
7. Documentazione JavaDoc	14
7.1 Stato della documentazione	14
7.2 Generazione della documentazione JavaDoc	14
8. Limiti della soluzione sviluppata	15
9. Sitografia e riferimenti	16

1. Installazione

1.1 Requisiti di sistema

Cine-Max è un'applicazione Java pura che non dipende da librerie esterne. I requisiti tecnici sono minimi:

Componente	Requisito
Java Development Kit (JDK)	Versione 21 (configurato in .classpath ed Eclipse)
Sistema operativo	Windows / macOS / Linux (multiplatforma)
IDE consigliato	Eclipse IDE for Java Developers
Dipendenze esterne	Nessuna

1.2 Setup ambiente Eclipse

Il progetto è configurato come progetto Eclipse Java standard. I file .project e .classpath nella radice del repository contengono tutta la configurazione necessaria.

Per importare il progetto in Eclipse:

1. File > Import > General > Existing Projects into Workspace
2. Selezionare la cartella radice del repository (CineMax-fixed o equivalente)
3. Il progetto Cine-Max apparirà nell'elenco: selezionarlo e cliccare Finish
4. Nella Run Configuration (tasto destro su CineMax.java > Run As > Run Configurations), nella tab Arguments impostare la Working directory su: \${workspace_loc:Cine-Max}

Questo passaggio è fondamentale: senza la working directory corretta, i percorsi relativi dei file CSV (data/utenti.csv, data/proiezioni.csv, data/prenotazioni.csv) non vengono risolti correttamente e il programma non troverà i dati.

1.3 Compilazione da riga di comando

Dalla cartella radice del progetto:

Windows

```
mkdir bin
javac -encoding UTF-8 -d bin ^
  src\cinemax\model\*.java ^
  src\cinemax\persistence\*.java ^
  src\cinemax\service\*.java ^
  src\cinemax\ui\*.java ^
  src\cinemax\CineMax.java
```

Linux / macOS

```
mkdir -p bin
javac -encoding UTF-8 -d bin \
  src/cinemax/model/*.java \
  src/cinemax/persistence/*.java \
```

```
src/cinemax/service/*.java \  
src/cinemax/ui/*.java \  
src/cinemax/CineMax.java
```

1.4 Creazione del JAR eseguibile

Creazione JAR

```
jar --create --file bin/CineMax.jar --main-class cinemax.CineMax -C bin .
```

1.5 Esecuzione

Esecuzione (dalla cartella radice del progetto)

```
java -jar bin/CineMax.jar
```

oppure, senza JAR:

```
java -cp bin cinemax.CineMax
```

2. Architettura dell'applicazione

2.1 Scelta architetturale: architettura a livelli

Cine-Max adotta un'architettura a tre livelli (three-tier architecture) che separa nettamente le responsabilità tra modello del dominio, logica applicativa e interfaccia utente. Questa scelta garantisce la coesione interna di ogni componente e la bassa dipendenza tra livelli diversi (principio di bassa dipendenza inter-livello).

Livello	Package	Responsabilit�
Model	cinemax.model	Classi di dominio pure: Film, Proiezione, Prenotazione, Utente e sottoclassi, Ruolo
Persistence	cinemax.persistence	Lettura e scrittura dei file CSV tramite repository dedicati
Service	cinemax.service	Logica applicativa: GestoreApp coordina model e persistence
UI	cinemax.ui	Interfaccia terminale (TUI): menu, input/output, navigazione

Il flusso delle dipendenze   unidirezionale: UI -> Service -> Model, con Persistence usato solo dal Service. Le classi del Model non dipendono da nessun altro livello.

2.2 Struttura del package cinemax

Struttura sorgenti

```
src/
cinemax/
  CineMax.java      <- punto di ingresso (main)
model/
  Utente.java       <- classe astratta base
  Cliente.java
  Bigliettaio.java
  Proiezionista.java
  Ruolo.java        <- enum
  Film.java
  Proiezione.java
  Prenotazione.java
persistence/
  FileManager.java  <- utilita I/O CSV
  UtenteRepository.java
  ProiezioneRepository.java
  PrenotazioneRepository.java
service/
  GestoreApp.java   <- service layer principale
  CinemaService.java <- service alternativo (in allineamento)
ui/
  MenuUI.java       <- utilita di stampa e input
  LoginUI.java
  GuestMenu.java
  ClienteMenu.java
  BigliettaioMenu.java
  ProiezionistaMenu.java
```

3. Strutture dati

3.1 Classi del Model layer

Gerarchia Utente

La classe Utente è astratta e definisce i dati comuni a tutti gli utenti del sistema. Ogni sottoclasse concreta rappresenta un ruolo specifico e implementa il metodo `getRuolo()` restituendo il valore dell'enum `Ruolo` corrispondente.

Classe	Ruolo
Utente (astratta)	Classe base: nome, cognome, username, passwordCifrata, dataNascita, luogoDomicilio
Cliente	Ruolo.CLIENTE — può effettuare prenotazioni
Bigliettaio	Ruolo.BIGLIETTAIO — può consultare prenotazioni
Proiezionista	Ruolo.PROIEZIONISTA — può gestire il palinsesto

L'identità di un utente è determinata dall'username, che deve essere univoco nel sistema. Il metodo `equals()` e `hashCode()` sono implementati coerentemente su questo criterio.

Film

La classe Film contiene esclusivamente i metadati dell'opera cinematografica: titolo, genere, regista, anno, durataMinuti, etaMinima. Non conosce quando o a quale prezzo viene proiettato: questa separazione permette di associare lo stesso film a più proiezioni senza duplicare i dati.

L'uguaglianza tra film è determinata dalla combinazione titolo + regista + anno.

Proiezione

La classe Proiezione associa un Film a una data/ora specifica con un prezzo. Ogni proiezione ha un codice UUID generato automaticamente alla creazione (o fornito esplicitamente per la ricostruzione da file).

La capienza della sala è una costante di classe:

```
Costante capienza  
public static final int CAPIENZA_SALA = 200;
```

Il numero di posti liberi non è memorizzato nella Proiezione ma calcolato a runtime dal `GestoreApp`, sottraendo il totale dei posti prenotati dalla capienza:

```
Calcolo posti liberi (GestoreApp)  
public int postiLiberi(Proiezione proiezione) {  
    int occupati = prenotazioni.stream()  
        .filter(p -> p.getProiezione().getCodice().equals(proiezione.getCodice()))  
        .mapToInt(Prenotazione::getNumeroPosti).sum();  
    return Proiezione.CAPIENZA_SALA - occupati;  
}
```

Prenotazione

La classe Prenotazione collega un Cliente a una Proiezione specificando il numero di posti. Il codice UUID è generato automaticamente; la data di prenotazione viene impostata all'istante corrente. Il costo totale non è memorizzato ma calcolato come `numeroPosti * proiezione.getPrezzoBiglietto()`.

3.2 Strutture dati in memoria nel GestoreApp

GestoreApp mantiene in memoria tre ArrayList che vengono caricati all'avvio e sincronizzati su file CSV ad ogni operazione di scrittura:

Attributi GestoreApp

```
private final List<Utente>    utenti    = new ArrayList<>();  
private final List<Proiezione> proiezioni = new ArrayList<>();  
private final List<Prenotazione> prenotazioni = new ArrayList<>();
```


4. Scelte algoritmiche

4.1 Cifratura delle password (SHA-256)

Le password non vengono mai memorizzate in chiaro. Al momento della registrazione o del login, la password viene convertita in un hash SHA-256 a 64 caratteri esadecimali usando la classe `MessageDigest` della libreria standard Java:

Metodo `hashPassword` in `GestoreApp`

```
private String hashPassword(String password) {
    MessageDigest digest = MessageDigest.getInstance("SHA-256");
    byte[] hash = digest.digest(password.getBytes(StandardCharsets.UTF_8));
    StringBuilder hex = new StringBuilder();
    for (byte b : hash) hex.append(String.format("%02x", b));
    return hex.toString();
}
```

Il login verifica le credenziali confrontando l'hash della password inserita con quello memorizzato. La password in chiaro non viene mai conservata.

4.2 Controllo sovrapposizione proiezioni

Prima di aggiungere o modificare una proiezione, il sistema verifica che non si sovrapponga temporalmente ad altre proiezioni già in programma. Il controllo tiene conto della durata del film e usa il classico algoritmo di intersezione di intervalli:

Metodo `siSovrapponeA` nella classe `Proiezione`

```
public boolean siSovrapponeA(Proiezione altra) {
    LocalDateTime inizioQuesta = this.dataOra;
    LocalDateTime fineQuesta = this.dataOra.plusMinutes(this.film.getDurataMinuti());
    LocalDateTime inizioAltra = altra.dataOra;
    LocalDateTime fineAltra = altra.dataOra.plusMinutes(altra.film.getDurataMinuti());
    return inizioQuesta.isBefore(fineAltra) && inizioAltra.isBefore(fineQuesta);
}
```

La condizione `inizioA < fineB && inizioB < fineA` e l'algoritmo standard per la rilevazione di sovrapposizione tra due intervalli: due proiezioni si sovrappongono se e solo se ognuna inizia prima che l'altra finisca.

4.3 Ricerca con filtri combinabili

La ricerca di proiezioni e prenotazioni è implementata tramite Java Streams con filtri concatenati. Ogni filtro è applicato solo se il criterio corrispondente è stato specificato (non nullo o non vuoto). Questo permette di combinare qualsiasi sottoinsieme di criteri in un'unica chiamata:

Ricerca proiezioni in `GestoreApp`

```
public List<Proiezione> cercaProiezioni()
```

```

String titolo, String genere,
LocalDate dataMin, LocalDate dataMax,
double prezzoMin, double prezzoMax) {
return proiezioni.stream().filter(p -> {
    Film f = p.getFilm();
    if (nonVuoto(titolo) && !f.getTitolo().toLowerCase()
        .contains(titolo.toLowerCase())) return false;
    if (nonVuoto(genere) && !f.getGenere()
        .equalsIgnoreCase(genere)) return false;
    LocalDate data = p.getDataOra().toLocalDate();
    if (dataMin != null && data.isBefore(dataMin)) return false;
    if (dataMax != null && data.isAfter(dataMax)) return false;
    if (prezzoMin >= 0 && p.getPrezzoBiglietto() < prezzoMin) return false;
    if (prezzoMax >= 0 && p.getPrezzoBiglietto() > prezzoMax) return false;
    return true;
}).collect(Collectors.toList());
}

```

La ricerca sul titolo usa `contains()` in modo case-insensitive, permettendo ricerche per titolo parziale come richiesto dalle specifiche.

4.4 Codici univoci con UUID

I codici identificativi di Proiezione e Prenotazione sono generati tramite `UUID.randomUUID().toString()`, che produce una stringa di 36 caratteri con probabilità di collisione trascurabile (2^{122} possibilità). Questi codici vengono persistiti nel CSV e riletti al riavvio senza essere rigenerati, grazie a costruttori specifici che accettano il codice come parametro.

5. Formato dei file CSV e gestione della persistenza

5.1 Formato dei file

La persistenza è realizzata su tre file CSV nella directory data/. Le righe che iniziano con # sono commenti e vengono ignorate in lettura.

utenti.csv

Formato

TIPO,nome,cognome,username,passwordHash,dataNascita,luogoDomicilio

Campo	Descrizione
TIPO	CLIENTE BIGLIETTAIO PROIEZIONISTA
nome, cognome	Dati anagrafici
username	Identificativo univoco per il login
passwordHash	Hash SHA-256 della password in chiaro (64 hex chars)
dataNascita	Formato yyyy-MM-dd, campo opzionale (può essere vuoto)
luogoDomicilio	Luogo di domicilio

proiezioni.csv

Formato

codice,titolo,genere,regista,anno,durataMinuti,etaMinima,dataOra,prezzoBiglietto

Campo	Descrizione
codice	UUID univoco della proiezione
titolo, genere, regista	Dati del film
anno	Anno di uscita del film (intero)
durataMinuti	Durata in minuti (intero, usato per il controllo sovrapposizioni)
etaMinima	Eta minima consigliata (0 = per tutti)
dataOra	Formato dd/MM/yyyy HH:mm
prezzoBiglietto	Prezzo in euro (decimale, separatore punto)

prenotazioni.csv

Formato

codice,usernameCliente,codiceProiezione,numeroPosti,dataPrenotazione

Campo	Descrizione
codice	UUID univoco della prenotazione
usernameCliente	Username del cliente che ha prenotato
codiceProiezione	UUID della proiezione associata
numeroPosti	Numero di posti prenotati (intero positivo)
dataPrenotazione	Formato dd/MM/yyyy HH:mm

5.2 Gestione dei percorsi

I percorsi dei file CSV sono relativi alla working directory da cui viene eseguito il programma, non alla posizione del JAR. I percorsi sono definiti come costanti nella classe CineMax:

Costanti di percorso in CineMax.java

```
private static final String PATH_UTENTI = "data/utenti.csv";  
private static final String PATH_PROIEZIONI = "data/proiezioni.csv";  
private static final String PATH_PRENOTAZIONI = "data/prenotazioni.csv";
```

Questa scelta impone che il programma venga sempre avviato dalla cartella radice del progetto. In Eclipse la working directory viene configurata nelle Run Configurations: \${workspace_loc:Cine-Max}.

5.3 Classe FileManager

La classe FileManager (package cinemax.persistence) fornisce le operazioni di I/O di basso livello usate dai repository:

- leggiRighe(String percorsoFile): legge tutte le righe non vuote e non commento, restituendo una List<String>. Usa BufferedReader con encoding UTF-8.
- scriviRighe(String percorsoFile, List<String> righe): sovrascrive il file con le righe fornite (apertura in modalità overwrite).
- aggiungiRiga(String percorsoFile, String riga): aggiunge una riga in append al file.
- dividiCampi(String riga): divide una riga CSV usando la virgola come separatore.
- costruisciRiga(String... campi): costruisce una riga CSV unendo i campi con virgola.
- verificaCampi(String[] campi, int minimo, String descrizione): lancia IllegalArgumentException se il numero di campi è inferiore al minimo atteso.

Ogni operazione di scrittura (salvataggio) avviene immediatamente dopo ogni modifica, garantendo la persistenza anche in caso di terminazione improvvisa del programma.

5.4 Strategia di persistenza nel GestoreApp

GestoreApp gestisce la persistenza in modo diretto, senza passare per i repository del package persistence (che sono in fase di allineamento all'API del model). I dati vengono caricati in memoria all'avvio tramite metodi privati (caricaUtenti(), caricaProiezioni(), caricaPrenotazioni()) e salvati su file tramite metodi privati corrispondenti (salvaUtenti(), salvaProiezioni(), salvaPrenotazioni()) chiamati al termine di ogni operazione di scrittura.

6. Pattern e scelte implementative

6.1 Pattern utilizzati

Singleton implicito — GestoreApp

GestoreApp agisce come punto centrale di accesso alla logica applicativa e allo stato del programma. Viene istanziato una sola volta nel main di CineMax e passato come parametro ai menu dell'interfaccia utente. Questo garantisce che esista un'unica sorgente di verità per lo stato dell'applicazione.

Factory implicita — costruttori specializzati

Le classi Proiezione e Prenotazione espongono due costruttori: uno che genera automaticamente il codice UUID (per la creazione di nuovi oggetti) e uno che accetta il codice come parametro esplicito (per la ricostruzione da file). Questo evita la rigenerazione dei codici durante il caricamento da CSV.

Classi utility non istanziabili

Le classi di interfaccia utente (MenuUI, LoginUI, GuestMenu, ClienteMenu, BigliettaioMenu, ProiezionistaMenu) e FileManager sono dichiarate final con costruttore privato, e i loro metodi sono tutti statici. Questo pattern rende esplicita la loro natura di collezioni di funzioni di supporto, senza stato proprio.

Template Method implicito — gerarchia Utente

La classe astratta Utente definisce il metodo astratto getRuolo(), che ogni sottoclasse implementa restituendo il proprio valore dell'enum Ruolo. Questo permette al codice del GestoreApp di determinare il ruolo di un utente tramite polimorfismo, senza usare instanceof nelle operazioni comuni.

6.2 Scelte implementative rilevanti

Enum Ruolo

I ruoli sono modellati come enum (CLIENTE, BIGLIETTAIO, PROIEZIONISTA), garantendo la sicurezza del tipo a compile time e la leggibilità del codice nei costrutti switch.

Uso di Java Streams

Le operazioni di ricerca e filtraggio sulle liste in memoria sono implementate tramite Java Streams, sfruttando i metodi filter(), map(), collect() e mapToInt(). Questa scelta rende il codice più dichiarativo e conciso rispetto ai cicli espliciti, a costo di una leggera complessità per chi non ha familiarità con la programmazione funzionale.

Encoding UTF-8 esplicito

Tutti i file CSV vengono letti e scritti con encoding esplicitamente impostato a UTF-8 (StandardCharsets.UTF_8), garantendo la corretta gestione di caratteri accentati e speciali su qualsiasi sistema operativo.

7. Documentazione JavaDoc

7.1 Stato della documentazione

Le classi del service layer e dell'interfaccia utente (package cinemax.service e cinemax.ui) sono documentate con JavaDoc completo: ogni classe, metodo e attributo significativo ha il corrispondente commento con descrizione, tag @param, @return e @throws dove applicabile.

Le classi del model layer (package cinemax.model), redatte da Pornima Macri, sono documentate con JavaDoc dettagliato per tutte le classi, costruttori, metodi e attributi.

7.2 Generazione della documentazione JavaDoc

Per generare la documentazione HTML dalla cartella radice del progetto:

Comando javadoc

```
javadoc -encoding UTF-8 -charset UTF-8 \  
-d doc/javadoc \  
-sourcepath src \  
-subpackages cinemax
```

La documentazione HTML viene generata nella cartella doc/javadoc/. Il punto di accesso è il file doc/javadoc/index.html.

8. Limiti della soluzione sviluppata

- Accesso concorrente non gestito: il sistema non implementa alcuna sincronizzazione. Accessi concorrenti da più istanze del programma sullo stesso filesystem possono causare corruzione dei file CSV.
- Nessuna validazione dei dati CSV: se i file CSV vengono modificati manualmente in modo non conforme al formato, il programma ignora le righe malformate senza segnalare l'errore all'utente finale.
- Persistenza non transazionale: un crash durante la scrittura di un file CSV può lasciare il file in uno stato parzialmente aggiornato.
- Scalabilità limitata: le ricerche avvengono con scansione lineare delle liste in memoria. Per dataset di grandi dimensioni si dovrebbe considerare l'uso di strutture indicizzate o un database.
- CinemaService non integrato: la classe CinemaService nel package service e un service layer alternativo ancora in fase di allineamento con l'API del model definitivo. L'applicazione usa GestoreApp come service layer principale.
- Nessun logging strutturato: i messaggi di errore interni sono stampati su System.err senza un framework di logging configurabile.

9. Sitografia e riferimenti

- Java SE 21 API Documentation — Oracle: <https://docs.oracle.com/en/java/javase/21/docs/api/>
- Java Streams API — Oracle:
<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/stream/Stream.html>
- MessageDigest (SHA-256) — Oracle:
<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/security/MessageDigest.html>
- UUID — Oracle: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/UUID.html>
- LocalDateTime / LocalDate — java.time — Oracle:
<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/time/package-summary.html>
- FIPS 180-4 (SHA-256 Standard) — NIST: <https://csrc.nist.gov/publications/detail/fips/180/4/final>
- Adoptium — Download JDK: <https://adoptium.net/>
- Specifiche progetto Laboratorio Interdisciplinare A, a.a. 2025/2026 — Giovanni Meroni, Università degli Studi dell'Insubria